重邮802数据结构押题卷（二）参考答案

对应临时区原《押题卷二.pdf》。按原卷题号给出答案、过程及必要配图；算法实现允许等价解法。树根层次取1，路径长度按边数计。题面或代码存在歧义时在题下注明。

一、程序阅读题

1．Prime函数（5分）

（1）**对n≥2判断n是否为素数**：检查2至floor(sqrt(n))是否有n的因子；有则返回0，无则返回1。若n为合数，必有一个不大于sqrt(n)的非平凡因子。

（2）最坏时间复杂度 **O(sqrt(n))**，辅助空间O(1)。小因子很早出现时可以提前结束。原代码对n=0或1也会返回1，实际使用须在开头补 if (n < 2) return 0;；负数不应进入sqrt计算。

2．折半查找填空（5分）

从上到下各空依次为：

```
= low + (high - low) / 2
high = mid - 1
low = mid + 1
```

题面空后已有分号。原文 K<[mid].key 缺数组名A，应为 K < A[mid].key；该if条件后的多余分号须删去，使后面的语句确为if分支。递增有序表上查找时间O(log n)，辅助空间O(1)。

3．栈的输出（5分）

按教材通常从左到右演算两次Pop，逐行输出：

```
15
22
8
5
2
10
```

先弹出4、再弹出3，x=4+2×3=10。随后数组按2、5、8、22、15依次入栈，所以最后按相反次序弹出，再输出10。

原码歧义：C++没有规定加号左右操作数的求值顺序，Pop(S)+2*Pop(S)中若先算右操作数，x=3+2×4=11，末行会变为11。为使题意唯一，应拆为 int p=Pop(S); int q=Pop(S); int x=p+2*q;。前五行不受影响。

4．alk函数功能（5分）

计算Ackermann函数A(m,n)，其递推定义为：A(0,n)=n+1；m>0时A(m,0)=A(m-1,1)；m,n>0时A(m,n)=A(m-1,A(m,n-1))。

例如A(1,n)=n+2，A(2,n)=2n+3，A(3,n)=2^(n+3)-3。函数增长极快；上述功能以数学整数且不发生机器整数溢出为前提。

二、填空题

1．由先中序求后序（3分）

E,C,G,F,D,H,B,A。根为A；其左子树根F，右子树根B，分别求得左后序E,C,G,F及右后序D,H,B。

2．堆删除的复杂度（3分）

O(log n)，通常指删除堆顶或已知位置的元素，用堆末元素替换后向上或向下调整。若只给关键字、须先在无索引的堆中定位任意元素，查找最坏O(n)，整个删除为O(n)。

3．树的度数和（3分）

n-1。树结点的度指孩子数，每条父子边恰贡献一次；n个结点的树有n-1条边。

4．后缀表达式求值（3分）

9。abc-/de*+对应a/(b-c)+d×e，代入得6/(4-2)+3×2=3+6=9。

5．由中后序求先序（3分）

a,b,d,e,c,f,g。根a，左子树根b且左右为d、e，右子树根c且左右为f、g。

6．折半查找比较次数（3分）

4；4。采用下中点时，两次均依次比较42、29、33、39；39命中，40在最后一次比较后区间变空。

7．二维数组地址（3分）

692。按行优先，设每行n列，则2n+2=676-644=32，得n=15，A[3][3]地址为644+3×15+3=692。按列优先同理得每列15个元素，结果相同。

8．三叉树度为2的结点数（3分）

4。设n2=n3=x，由叶数n0=1+n2+2n3，得13=1+3x。

9．BST的中序（3分）

18,27,36,39,43,44,77。二叉搜索树的中序遍历按关键字递增。

10．20结点AVL的高度（3分）

最大6，最小5。AVL高度h所需最少结点N(h)=1+N(h-1)+N(h-2)，N(0)=0,N(1)=1；
N(5)=12,N(6)=20,N(7)=33，所以最大高6。容量满足2^4-1<20≤2^5-1，且5层的AVL可容纳20个结点，故最小高5。

三、综合应用题

1．所有拓扑有序序列（10分）

答案：所有可能的拓扑有序序列共有 **4种**。

序号	拓扑有序序列
1	1，2，3，5，4，6
2	1，2，5，3，4，6
3	1，2，5，4，3，6
4	1，3，2，5，4，6

计算过程：由原图可读出弧集为{1→2，1→3，1→4，2→4，2→5，5→4，5→6，4→6，3→6}。初始入度如下。

顶点	1	2	3	4	5	6
入度	0	1	1	3	1	3

顶点1必须首先输出，顶点6必须最后输出。其余顶点中，2、5、4的相对次序固定为2→5→4，而3可以插在这三个顶点之前、之间或之后，因此恰有上述4种序列。每种序列的逐步可选顶点如下；每个集合列出该步输出前所有尚未输出且入度为0的顶点。

输出序列	第1步	第2步	第3步	第4步	第5步	第6步
1，2，3，5，4，6	{1}	{2,3}	{3,5}	{5}	{4}	{6}
1，2，5，3，4，6	{1}	{2,3}	{3,5}	{3,4}	{4}	{6}
1，2，5，4，3，6	{1}	{2,3}	{3,5}	{3,4}	{3}	{6}
1，3，2，5，4，6	{1}	{2,3}	{2}	{5}	{4}	{6}

若拓扑排序能够顺利输出全部顶点，则说明该有向图**无有向环，即为有向无环图（DAG）**。

2．通讯网的全部最省线路方案（10分）

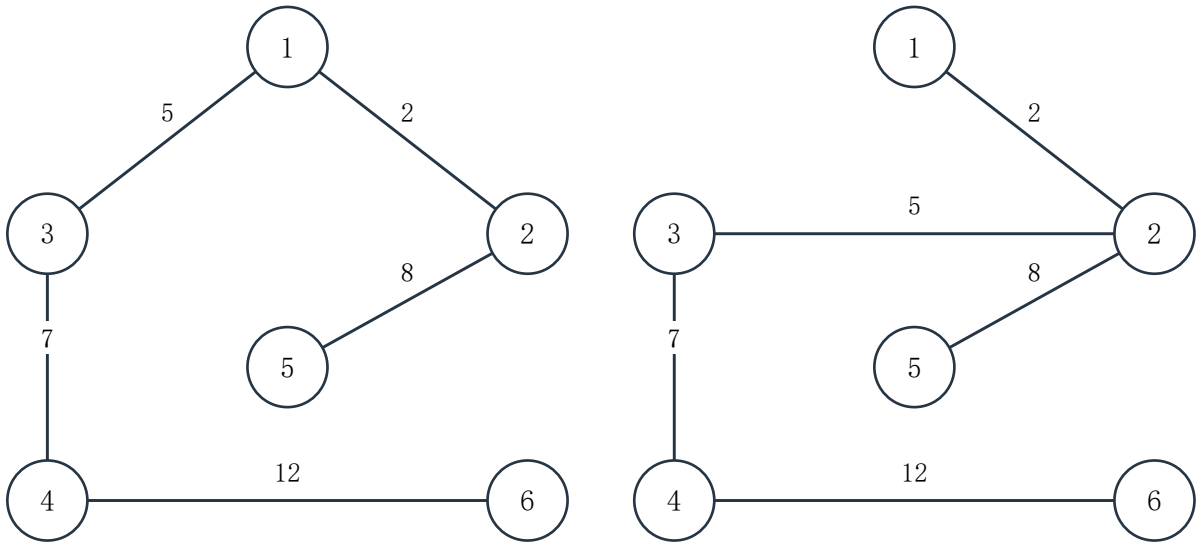
答案：该问题等价于求最小生成树，共有 **2种**最省线路方案，每种均选5条线路，总代价为 **34**。

方案	所选线路及代价	总代价
方案一	1-2(2)，1-3(5)，3-4(7)，2-5(8)，4-6(12)	34
方案二	1-2(2)，2-3(5)，3-4(7)，2-5(8)，4-6(12)	34

计算过程：按Kruskal算法从小到大考察边。1-3和2-3的权值同为5，只能选其中一条；另一条与已选边1-2形成三角形，应舍去。除此处外，后续选边相同。

考察的边	权值	处理	已选边累计代价
1-2	2	选择	2
1-3或2-3	5	二选一，连接顶点3	7
另一条权5的边	5	与前两条边构成环，舍去	7
3-4	7	选择	14
2-5	8	选择	22
3-5	9	形成环，舍去	22
4-5	10	形成环，舍去	22
4-6	12	选择；已得5条边，结束	34

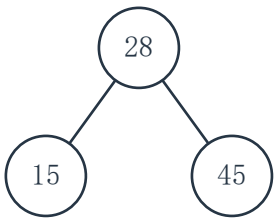
因此，公共边为{1-2，3-4，2-5，4-6}，再从{1-3，2-3}中任选一条，便得到全部最小生成树。总费用均为2+5+7+8+12=34。



全部两种最小生成树：左为方案一，右为方案二，总代价均为34

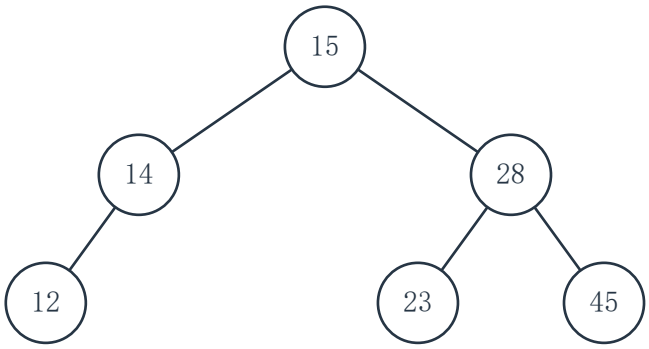
3．AVL插入与调整（10分）

（1）图(a)插入15，成为28的左孩子。结点45出现LL失衡，对45作一次右旋，结果如下。



（a）插入15并作LL调整后

（2）图(b)查找路径为28→14→15，23插为15的右孩子。最低失衡结点是28，属于LR型；先对14左旋，再对28右旋。最终为：

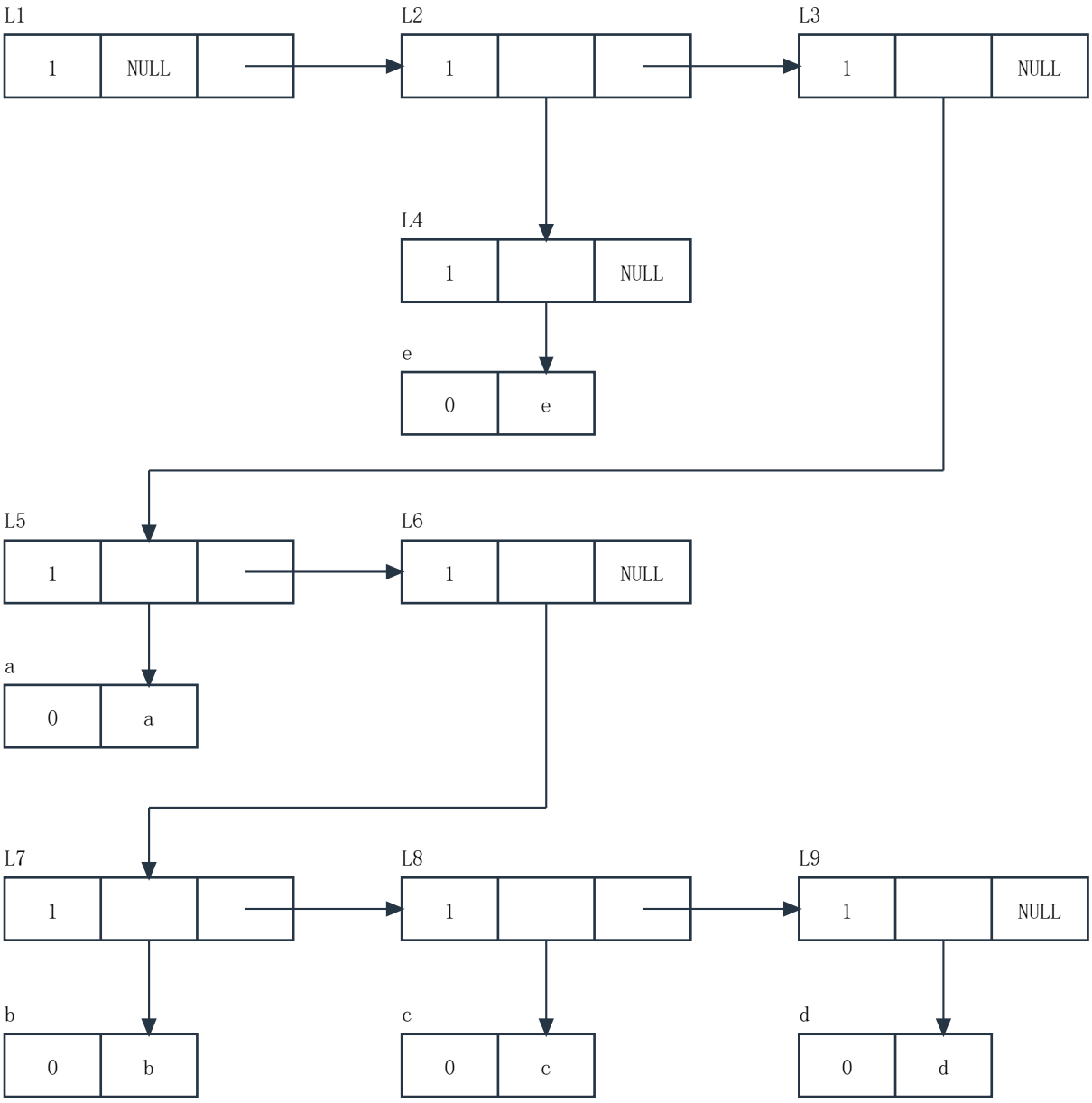


（b）插入23并作LR调整后

其孩子关系为15:(14,28)、14:(12,空)、28:(23,45)。中序为12,14,15,23,28,45，各结点左右高度差绝对值均不超过1。

4．广义表的头尾链表（10分）

结构约定：表结点画成[1|hp|tp]，hp指表头元素、tp指余下表；原子结点画成[0|值]。空表用NULL指针表示，不建立原子结点。LS指向L1。



LS=(( ),(e),(a,(b,c,d)))的头尾链表；箭头分别从hp或tp域引出

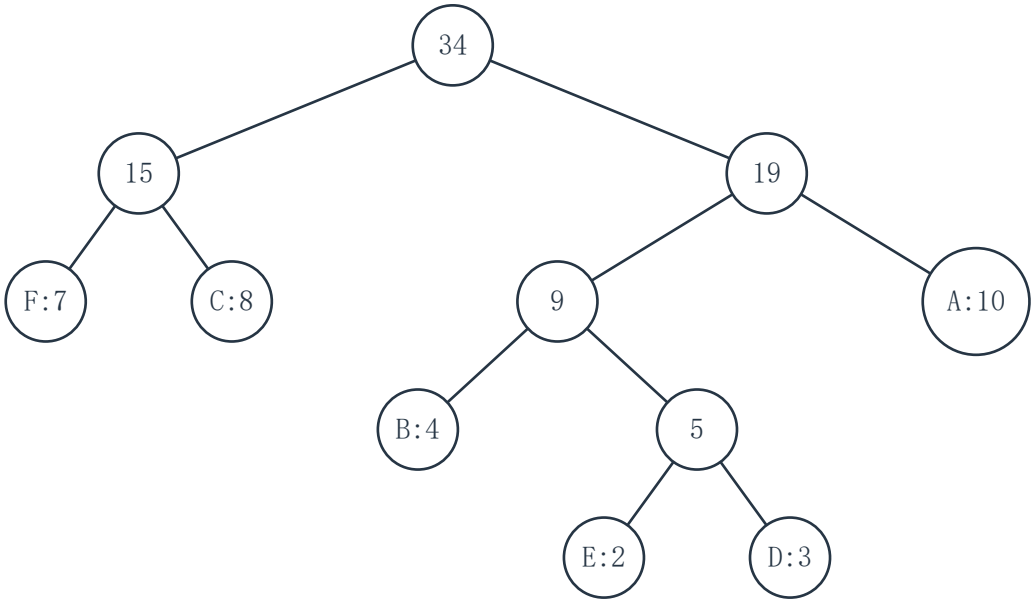
根层的L1、L2、L3依次表示三个元素：空表()、子表(e)、子表(a,(b,c,d))。因此L1.hp=NULL，但L1本身存在；这与整个LS为空表不同。图中L6.hp指向(b,c,d)的首个表结点L7，L6.tp=NULL。

核对：L1→L2→L3是最外层tp链；L5→L6是(a,(b,c,d))的tp链；L7→L8→L9是(b,c,d)的tp链。原子结点e、a、b、c、d分别由相应表结点的hp指向。

5．Huffman树与编码（10分）

合并过程：

轮次	选取的两个最小权值	新权值
1	E(2)、D(3)	5
2	B(4)、5	9
3	F(7)、C(8)	15
4	9、A(10)	19
5	15、19	34



一组Huffman树（左边为0、右边为1，叶标字符:权值）

取左0右1，一组编码为：

字符	A	B	C	D	E	F
权值	10	4	8	3	2	7
编码	11	100	01	1011	1010	00

校验WPL=10×2+4×3+8×2+3×4+2×4+7×2=82，也等于5+9+15+19+34=82。题目未限定左右方向，交换左右子树所得其他最优前缀编码也正确。

6．线性探查散列表（10分）

（1）答案：取H(key)=key mod7，线性探查地址为H_i(key)=[H(key)+i] mod7，i=0,1,…。最终散列表如下。

下标	0	1	2	3	4	5	6
关键字	55	22	63	31	空	26	13

插入过程：

关键字	初始地址	依次探查的地址	最终地址	探查次数
13	6	6	6	1
22	1	1	1	1
31	3	3	3	1
55	6	6→0	0	2
26	5	5	5	1
63	0	0→1→2	2	3

例如，55的初始地址为6，该位置已有13，故回绕探查地址0并存入；63的初始地址为0，依次遇到55和22，最终存入地址2。

（2）模数的选择：除留余数法一般选取不大于表长、且接近表长的素数作为模数，以减少某些关键字分布规律造成的冲突。本题表长为7，7本身是素数，故取模7。

（3）删除方法：采用“删除标记”法。每个位置设置三种状态：从未使用的空位置EMPTY、已有元素的位置OCCUPIED、已删除的位置DELETED。删除时将该位置改为DELETED，查找遇到DELETED仍继续探查；只有遇到EMPTY或探查完整个表，才能判定查找失败。插入时可以复用DELETED位置。

不能把删除位置直接改为空。例如，13位于地址6，55因冲突存于地址0。若删除13后把地址6直接置为EMPTY，再查找55时会在初始地址6提前结束，错误地报告不存在。保留DELETED标记即可维持原探查链。

7．Prim最小代价生成树（10分）

题面说明：原图中心顶点与右上顶点均标为“2”。为区分两个不同顶点，以下保留右上顶点为2，临时将中心顶点记为 **X**；图中的边及权值均保持原意。

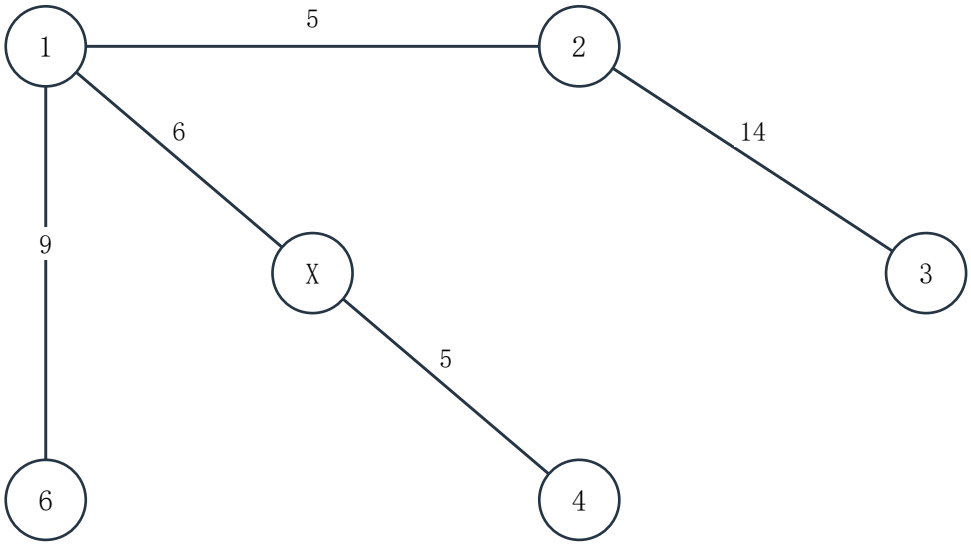
答案：以顶点1开始，一种Prim选边顺序为1-2、1-X、X-4、1-6、2-3，对应的最小生成树代价为 **39**。

计算过程：令U为已加入生成树的顶点集合。表中“w(v)”表示当前从U连接该列顶点的最小边权为w、所接U中顶点为v；“-”表示该顶点已加入U。若候选边等权，保留先前记录的连接边。

已加入的顶点集合U	顶点2	顶点X	顶点6	顶点4	顶点3
{1}	5(1)	6(1)	9(1)	∞	∞
{1,2}	-	6(1)	9(1)	12(2)	14(2)
{1,2,X}	-	-	9(1)	5(X)	14(2)
{1,2,X,4}	-	-	9(1)	-	14(2)
{1,2,X,4,6}	-	-	-	-	14(2)
{1,2,X,4,6,3}	-	-	-	-	-

步骤	新加入的顶点	选中的边	边权	累计代价
1	2	1-2	5	5
2	X	1-X	6	11
3	4	X-4	5	16
4	6	1-6	9	25
5	3	2-3	14	39

最小生成树的边集为 **{1-2，1-X，X-4，1-6，2-3}**，代价为5+6+5+9+14=39。连接顶点6时，边X-6与1-6的权值均为9；若取X-6替代1-6，也得到一棵代价为39的最小生成树。



Prim所得一棵最小生成树，总代价39（X为原图中央顶点）

四、算法设计与分析题

1．双亲数组求层次和树高（10分）

基本思想：设parent[i]存结点i的双亲下标，根的双亲记为-1；输出数组c初始化为0，表示层次尚未计算。递归求某结点层次时，先求双亲层次再加1；已计算的结果直接复用。无须假定双亲总排在孩子前面。所有层次的最大值就是树高。

C语言实现：

```
static int getLevel(int i, const int parent[], int c[]) {
    if (c[i] != 0) return c[i];
    if (parent[i] == -1) c[i] = 1;
    else c[i] = getLevel(parent[i], parent, c) + 1;
    return c[i];
}
int levelsAndHeight(const int parent[], int n, int c[]) {
    for (int i = 0; i < n; ++i) c[i] = 0;
    int height = 0;
    for (int i = 0; i < n; ++i) {
        int h = getLevel(i, parent, c);
        if (h > height) height = h;
    }
    return height;
}
```

复杂度：每个结点的c值只求一次，时间O(n)；递归栈最大O(h)，最坏O(n)。c为题目指定的输出数组，占O(n)；若计入输出空间，总空间O(n)。假定输入为合法的树；n=0时返回高度0。逐个结点独立上溯而不复用层次，最坏会耗时O(n^2)。

2．递归求Huffman树的WPL（10分）

基本思想：从根的深度0开始递归，仅在叶结点计算“权值×深度”；内部结点的element不计入WPL。子树深度每深入一条边加1，累加左右结果。

C语言实现：

```
#include <stddef.h>
typedef struct HNode {
    int element;
    struct HNode *lchild, *rchild;
} HNode;
static long long weightSum(const HNode *p, int depth) {
    if (p == NULL) return 0;
    if (p->lchild == NULL && p->rchild == NULL)
        return (long long)p->element * depth;
    return weightSum(p->lchild, depth + 1) +
           weightSum(p->rchild, depth + 1);
}
long long WPL(const HNode *T) {
    return weightSum(T, 0);
}
```

复杂度：每个结点访问一次，时间O(n)；递归栈O(h)，最坏O(n)。空树返回0；仅一个叶结点的树，其根深度为0，按带权路径长度的数学定义返回0。此处不额外约定单字符编码至少占1位。

3．有序单链表插入（10分）

基本思想：假设原表按element非递减有序，L为头结点。从头结点起找第一个element≥x的结点，将新结点插在其前面；若不存在则在表尾插入。新结点的后继先指向原后继，再修改前驱的link。

C语言实现：

```
#include <stdlib.h>
typedef struct Node {
    int element;
    struct Node *link;
} Node;
void insertSorted(Node *L, int x) {
    Node *pre = L;
    while (pre->link != NULL && pre->link->element < x)
        pre = pre->link;
    Node *s = malloc(sizeof *s);
    if (s == NULL) abort();
    s->element = x;
    s->link = pre->link;
    pre->link = s;
}
```

复杂度：最坏扫描全部n个数据结点，时间O(n)；除新插入的结果结点外，辅助空间O(1)。空表、表头前插、表尾后插和等值元素均能处理；若原表为非递增，循环中的 < x 改为 > x。